



GPU KERNELS IN DOLFINx

Chris Richardson, Joe Dean, Garth Wells

University of Cambridge

Submission type: Poster

Presenter: Chris Richardson (University of Cambridge)

There are many different ways that we can think about using GPUs for Finite Element Modelling. Over the last few years, along with others, we have been working on various different projects that use GPUs with FEniCSx. The simplest approach is to use the existing CPU code for matrix assembly, and copy the sparse matrix to the GPU. It doesn't require much (or sometimes any) code changes: we rely on the solver to handle the details. This approach can be used with PETSc/hypre, Ginkgo, amgX and CUDSS (NVIDIA's direct solver). Despite this simplicity, we are still faced with the issue of binding CPU to GPU - at the moment this is done by one CPU core to one GPU, which makes the mesh processing relatively slow. We are developing threading techniques on CPU to speed this up. The next level of complexity involves on-device assembly: first, we need to prepare a sparsity pattern on the device, then accumulate into that sparse data structure. Since ffcx-generated code is not very GPU-friendly, we write custom code, pre-computing geometric factors, which can be streamed from memory. For higher-order problems, matrices are not efficient, and we have developed some matrix-free methods for p-multigrid, which can run in parallel across a HPC system with hundreds of GPU devices to solve very large problems. There is also some work on Python wrappers for GPU data, that allow interoperability with cupy arrays, which works for both NVIDIA and AMD GPUs.

References

DOI: 10.1016/j.parco.2023.103051, <https://scientificcomputing.github.io/fenics2024/gpu-acceleration>, DOI: 10.1016/j.procs.2025.08.235